A Comparative Analysis of Energy Efficiency Across Large Language Models

Fatin Israq Talha, Mehrab Sadekin Borno, Md Rakib Hasan Rabbi, Muhmmad Ahamadul Hasan Prianto

Department of Computer Science and Engineering
United International University, Dhaka, Bangladesh

Abstract

The rapid adoption of large language models has led to a common but energy-inefficient practice: routing all user prompts, regardless of complexity, through the same large-scale model. This uniform approach incurs substantial and often unnecessary energy costs, as many prompts do not require the full capacity of a large model to be answered accurately. In this work, we first conduct an empirical study measuring the energy consumption and output quality of multiple open-source language models across varying sizes and quantization levels, using real-time GPU power measurements. Building on these measurements, we propose a lightweight prompt-complexity router that dynamically selects the smallest capable model for a given input, reserving larger models only for prompts that genuinely require them. We evaluate our system in terms of total energy consumption, response quality, and latency, comparing it against a baseline that always uses a single large model. Our results demonstrate that adaptive model routing can substantially reduce energy consumption with minimal degradation in answer quality, offering a practical and generalizable strategy for building more sustainable AI-serving systems.

Keywords: Large Language Models, Energy Efficiency, Green Computing, Model Quantization, Model Routing, Sustainable AI

1 Introduction

The past several years have seen an unprecedented surge in the adoption of Large Language Models (LLMs) across consumer applications, enterprise software, and scientific research. Models such as GPT, Llama, and Mistral now handle billions of inference requests daily, ranging from simple factual lookups to complex multi-step reasoning and code generation tasks. This growth in usage has been accompanied by an equally significant growth in the computational and energy resources required to serve these models, a trend that has drawn increasing scrutiny from the Green AI and sustainable computing research communities [1, 2]

A large share of this energy cost stems from a simple but pervasive architectural inefficiency: most LLM-serving systems route every incoming prompt to the same model, regardless of the prompt's actual complexity. A one-word sentiment classification query and a multi-step mathematical reasoning problem are typically processed by identical model capacity, even though the former could plausibly be answered by a model an order of magnitude smaller and cheaper to run. This uniform, "one-size-fits-all" serving strategy is computationally wasteful, and at scale, the aggregate energy cost of this inefficiency is substantial — prior work has shown that training and serving large NLP models can carry carbon and water footprints comparable to the lifetime emissions of multiple automobiles or the water consumption of thousands of households [1].

While this problem is increasingly well recognized, existing research has generally approached it from three largely separate angles. The first body of work focuses on *measurement*: profiling the real-world energy and power draw of LLM inference on GPU hardware to establish reliable baselines [3–7]. The second focuses on *model compression*, using quantization techniques to reduce a single model's memory footprint and energy consumption while attempting to preserve accuracy [8–11] The third focuses on *routing and cascading*, in which a lightweight decision mechanism dynamically selects which model — from a pool of models of varying size and capability — should handle a given query, escalating to a larger model only when necessary [12–15]. Each of these strands has produced valuable results in isolation, but they are rarely combined: measurement studies rarely inform a live serving-time decision, quantization studies rarely consider dynamically selecting among multiple differently-sized models, and routing studies are almost

always optimized for monetary cost, latency, or answer quality - with energy usage treated as an afterthought or estimated indirectly, rather than measured directly from GPU power draw during inference.

This disconnect represents a meaningful gap in the literature: there is currently no system in the reviewed body of work that simultaneously (i) measures LLM inference energy directly using real-time GPU power sampling, (ii) treats model quantization level as a first-class variable in the model-selection decision, and (iii) makes that selection decision per-prompt, based on an estimate of the prompt's complexity, at serving time. Closing this gap is the central motivation of the present work.

To this end, this paper makes the following contributions:

- We conduct an empirical energy-profiling study across multiple open-source language models spanning different parameter scales and quantization levels, using real-time GPU power measurements captured via NVML on consumer-grade hardware.
- Building on these measurements, we design and implement a lightweight, prompt-complexity-aware router that dynamically selects the smallest capable model — from a tiered pool of quantized model candidates — for each incoming query, reserving larger models for prompts that genuinely require additional reasoning capacity.
- We evaluate the resulting adaptive-routing system against a static, single-large-model baseline along three dimensions: total energy consumption, output quality retention, and end-to-end latency, using a curated benchmark workload spanning low-, medium-, and high-complexity task categories.

Our results demonstrate that adaptive, complexity-aware model routing can substantially reduce the energy footprint of LLM inference while incurring only minimal degradation in output quality relative to always using the largest available model. In doing so, this work offers a practical, hardware-grounded, and generalizable strategy for building more energy-efficient and environmentally sustainable LLM-serving systems — directly addressing the measurement–routing–quantization gap identified in the literature review that follows.

2 Related Works

2.1 Literature Review

The growing energy footprint of large language model (LLM) inference has motivated a substantial body of work that can be organized into four broad, interconnected strands: (i) empirical measurement of LLM inference energy, (ii) quantization as a means of reducing the energy-accuracy trade-off, (iii) model routing and cascading strategies that avoid invoking large models unnecessarily, and (iv) foundational work establishing the carbon and environmental footprint of deep learning and motivating the broader Green AI agenda.

A first line of work focuses on directly profiling the energy consumed during LLM inference under real hardware conditions. Samsi et al. [3] benchmark the energy costs of several open-source LLMs on GPU infrastructure, establishing baseline power and latency figures that later routing and quantization studies build upon. Fernandez et al. [4] extend this measurement effort by explicitly considering efficiency optimizations, framing energy consumption as a first-class metric alongside accuracy and latency. Complementing these single-node measurement studies, Wilkins et al. [5] show that heterogeneous GPU clusters can be scheduled to lower the aggregate energy consumption of inference workloads, suggesting that energy savings can be achieved not only at the model level but also at the infrastructure-scheduling level. Ji and Jiang [6] provide a systematic review of electricity demand associated with LLMs, consolidating evaluation methodologies and open challenges across the field. Most recently, Niu et al. [7] introduce TokenPowerBench, a benchmark that captures GPU-, node-, and system-level power draw and attributes energy separately to the prefill and decode phases of inference, offering fine-grained evidence that informs the design of measurement pipelines such as the one used in this project.

A second strand investigates quantization as a direct lever for reducing the energy cost of inference while attempting to preserve output quality. Frantar et al. [8] introduce GPTQ, a post-training quantization method that compresses model weights with minimal accuracy loss, and Lin et al. [16] propose AWQ, which protects a small subset of salient weights to further improve the accuracy-compression trade-off. Xiao et al. [10] address the difficulty of quantizing activations by smoothing outlier values prior to quantization, and Dettmers et al. [11] demonstrate that 8-bit matrix multiplication can match full-precision performance at large model scale. Collectively, this strand establishes that quantization level is a controllable variable that trades a measurable amount of accuracy for a measurable amount of energy savings – precisely the trade-off this project seeks to characterize empirically.

A third strand, closely aligned with the routing component of this project, explores directing individual prompts to different models based on estimated complexity, rather than applying quantization uniformly. Chen et al. [12]

propose FrugalGPT, a cascading strategy that queries progressively larger LLMs only when earlier, cheaper models are deemed insufficiently confident, substantially reducing cost while retaining accuracy. Ong et al. [13] learn a routing function from human-preference data to decide, per query, whether a smaller or larger model should respond. Ding et al. [14] formalize a similar problem as cost-efficient, quality-aware query routing between a small and a large model. Khan et al. [15] present a case study connecting routing-style optimizations to energy-efficiency metrics specifically, rather than cost alone. While these works demonstrate that routing can reduce inference cost or latency, comparatively few directly measure the resulting energy savings using real-time hardware power measurements, which is a gap this project's proposed router is designed to address.

Finally, a foundational strand of work established the environmental case for energy-aware AI system design in the first place. Strubell et al. [1] were among the first to quantify the carbon emissions of training large NLP models, prompting widespread attention to the sustainability of deep learning research. Schwartz et al. [2] formalize the notion of "Green AI," arguing for efficiency as an explicit research objective alongside accuracy. Li et al. [9] draw attention to the often-overlooked water footprint of AI model training and serving. Luccioni et al. [17] provide a detailed carbon-footprint case study of the 176B-parameter BLOOM model, illustrating the scale of emissions associated with large-model deployment. Together, these works motivate the broader problem this project addresses: reducing the operational energy footprint of LLM inference through model-level and system-level design choices.

2.2 Gap Analysis

Table 1 summarizes the reviewed literature according to problem definition, contribution, methodology, and the specific gap that motivates the present work. Across the reviewed studies, three recurring gaps emerge. First, energy-measurement studies [3–7] establish sound measurement methodology but rarely combine those measurements with an actionable serving-time decision mechanism. Second, quantization studies [8, 10, 11, 16] characterize the energy-accuracy trade-off of compressing a single model but do not consider dynamically choosing among multiple models of different sizes per prompt. Third, routing and cascading studies [12–15] optimize primarily for monetary cost or latency, with energy treated as, at best, a secondary or estimated metric rather than one measured directly from GPU power draw. This project is positioned to close these gaps by combining direct, real-time GPU energy measurement (as in the first strand) with a prompt-complexity router (as in the third strand) that additionally accounts for quantization level (as in the second strand) when selecting a model, and by evaluating the resulting system explicitly on measured energy consumption rather than proxy cost metrics.

As summarized above, no reviewed work simultaneously (1) measures LLM inference energy directly via real-time GPU power draw, (2) incorporates model quantization level as a routing variable, and (3) makes a per-prompt, complexity-aware model-selection decision at serving time. This combined gap is the primary motivation for the energy-aware prompt-complexity router proposed in this work.

Table 1: Gap analysis of the papers most closely related to this project.

Ref.	**Problem Definition**	**Contribution**	**Methodology**	**Gap Found**
[3]	There is no common benchmark for how much energy popular LLMs actually use.	Benchmarked the energy cost of several open-source LLMs on GPUs.	Measured GPU power draw while models processed a fixed set of prompts.	Gives useful energy numbers but doesn't decide which model to use for a given task.
[7]	Existing energy tools focus on training or overall inference performance, without a lightweight way to attribute power specifically within inference.	TokenPowerBench: an open-source benchmark measuring GPU-, node-, and system-level power, split by prefill and decode phase.	Ran multiple open-source model families (up to 405B params) under varying batch size, context length, and quantization; measured joules/token.	Provides a reusable, phase-aware energy-measurement framework, but doesn't use the measurements to pick a model or quantization level per prompt.
[12]	Sending every question to a large, expensive model wastes money.	FrugalGPT: try cheaper models first, only escalate to a bigger model if needed.	Cascades models and checks confidence before moving to a bigger one.	Built to save money and keep accuracy up – not built around measured energy use.

Continued on next page

Table 1: Gap analysis of the papers most closely related to this project.
(*Continued*)

Ref.	Problem Definition	Contribution	Methodology	Gap Found
[13]	Need an automatic way to decide which model should answer a given query.	RouteLLM: a router trained on human-preference data to pick small vs. large models.	Trains a classifier that routes each query to a small or large model.	Optimized for answer quality and cost, not for measured energy savings.
[15]	Many LLM optimization studies don't clearly connect their techniques to energy savings.	A case study linking several optimization techniques to energy-efficiency outcomes.	Evaluated existing optimization techniques against energy and performance metrics.	Looks back at existing techniques rather than building a live system that routes prompts to save energy.

3 Methodology

This section details the experimental methodology developed to systematically investigate and leverage the energy-quality trade-offs inherent in large language model inference. The proposed framework integrates three core components: an adaptive serving architecture backed by a tiered, quantized model pool; a lightweight, low-overhead prompt-complexity routing mechanism; and a multi-platform hardware testbed coupled with real-time GPU power instrumentation and multi-dimensional evaluation metrics.

3.1 System Architecture and Quantized Model Tiers

The overarching system architecture, illustrated in Figure 1, is designed to eliminate the computational and energetic waste of uniform model serving by dynamically matching query complexity to model capacity. When an incoming user prompt arrives, it is first intercepted at the host level where syntactic and semantic features are extracted to determine task difficulty. The request is then dispatched to the smallest capable model tier residing in GPU memory. During subsequent autoregressive token generation, an isolated background profiler records real-time power draw to compute precise energy consumption, after which the generated response, latency profile, and power statistics are aggregated for evaluation.

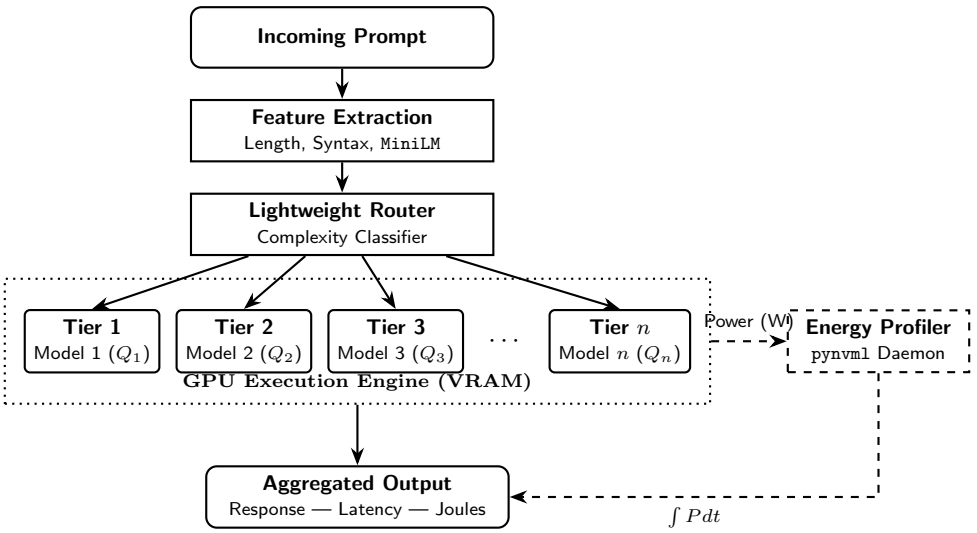

Figure 1: Generic system architecture pipeline demonstrating the adaptive routing mechanism across n candidate model tiers and real-time energy profiling.

To replicate practical, cost-effective deployments on consumer workstations and edge servers, the candidate model pool is structured to operate under a strict 8 GB VRAM budget. To maximize the parameter diversity and reasoning capability available within this memory envelope, we organize the model candidate pool into three distinct operational tiers, pairing specific model architectures with tailored post-training quantization schemes via the GGUF container format:

Tier 1 (Lightweight / Low Complexity): Designed for minimal active power draw and rapid execution, Tier 1 deploys compact parameter architectures, specifically `Llama-3.2-1B-Instruct`. Because the base parameter count is small, the model is served using 8-bit quantization (`Q8_0`), demanding approximately 1.3 GB of VRAM. Retaining 8-bit precision preserves near-lossless token representation and semantic fidelity on low-complexity tasks—such as binary sentiment classification, intent recognition, and short factual extraction—while executing with minimal computational arithmetic and abbreviated GPU active states.

Tier 2 (Intermediate / Medium Complexity): Serving as the mid-tier workhorse, Tier 2 incorporates mid-sized models, primarily `Llama-3.2-3B-Instruct` and `Phi-3.5-mini-3.8B`. These models are quantized using 4-bit medium block-level quantization (`Q4_K_M`), yielding an active memory footprint between 2.6 GB and 3.1 GB of VRAM. This tier provides balanced contextual reasoning, multi-paragraph coherence, and structured entity transformation suitable for abstractive summarization, natural language inference, and JSON restructuring, without incurring the substantial power draw of full-scale models.

Tier 3 (Heavyweight / High Complexity): Tier 3 deploys high-capacity open-weight foundation models, specifically `Llama-3.1-8B-Instruct` and `Mistral-7B-Instruct-v0.3`. Operating under `Q4_K_M` 4-bit quantization, these models consume between 5.2 GB and 5.8 GB of VRAM, remaining strictly within the 8 GB physical threshold. Tier 3 is reserved exclusively for complex prompts requiring multi-step mathematical derivation, chain-of-thought logic deduction, and algorithmic code synthesis.

All model candidates are executed using optimized C/C++ inference runtimes (`llama.cpp`), enabling direct memory-mapped model weights, zero-copy buffer allocations, and efficient CUDA kernel execution across heterogeneous hardware.

3.2 Prompt Complexity Estimation and Routing Mechanism

The primary objective of the prompt router is to predict the minimal capable model tier for any incoming query prior to full model invocation[cite: 3]. A fundamental design constraint is that the computational overhead of the routing decision—both in execution latency (Δt_{route}) and consumed energy (ΔE_{route})—must be negligible compared to the primary inference pass[cite: 3]. If routing overhead were significant, the energy savings achieved by directing queries to smaller models would be diminished[cite: 3].

To maintain ultra-low latency and avoid GPU resource contention, feature extraction is offloaded entirely to host CPU cores[cite: 3]. This architectural separation prevents GPU context-switching penalties, CUDA stream synchronization stalls, and VRAM memory fragmentation[cite: 3]. The router extracts a hybrid feature representation $x \in \mathbb{R}^d$ that fuses two complementary information modalities[cite: 3]:

- **Lexical and Syntactic Heuristics:** Surface-level heuristics capture explicit structural signals, including total prompt token length, character count, average word length, punctuation density, and boolean keyword flags targeting programmatic syntax (such as `def`, `class`, `import`, `return`, `SELECT`, and `curl`)[cite: 3].
- **Semantic Embeddings:** To capture latent linguistic complexity and semantic intent beyond surface keywords, the router computes a dense 384-dimensional sentence embedding using the compact `all-MiniLM-L6-v2` transformer model (22 million parameters)[cite: 3]. Executed on host CPU cores via the ONNX Runtime, this encoder constructs semantic representations in under 3 ms per query without consuming GPU VRAM[cite: 3].

Classification Algorithms and Decision Logic

The concatenated feature vector is fed into a trained multi-class classifier[cite: 3]. We evaluate two low-latency classification backends: an L_2-regularized multinomial Logistic Regression model and a shallow Random Forest ensemble[cite: 3]. The classifier produces posterior probability distributions $\hat{p}_i = P(y = \text{Tier } i|x)$ across the candidate tiers $i \in \{1, 2, 3\}$[cite: 3]. The final model tier selection is determined via an argmax decision rule $\hat{y} = \arg\max_i \hat{p}_i$ with configurable confidence escalation thresholds (τ_1, τ_2) to bias routing conservatively toward higher tiers whenever classification uncertainty is elevated[cite: 3].

Classifier Training and Threshold Calibration

Because both the Logistic Regression and Random Forest backends require supervised learning, we construct a dedicated training dataset independent of our final 600-prompt evaluation benchmark. We sample 3,000 diverse

prompts from the training splits of our selected datasets (SST-2, SQuAD, CNN/DailyMail, MNLI, GSM8K, and HumanEval). Ground-truth tier labels $y \in \{1, 2, 3\}$ are established empirically: a prompt is assigned to the lowest model tier capable of generating a correct response (verified via Exact Match, ROUGE-L, or execution accuracy).

The classifiers are trained using an 80-20 train-validation split. To select the final routing backend, we utilize 5-fold cross-validation on the training set, optimizing for a latency-constrained macro-F_1 score. The selected model must strictly maintain a sub-5 ms inference latency while maximizing classification accuracy[cite: 3].

Finally, the confidence escalation thresholds (τ_1 for Tier 1 to Tier 2, and τ_2 for Tier 2 to Tier 3) are calibrated on the 20% validation split using a grid search to minimize the False Negative Rate (FNR) of complexity estimation. A false negative—routing a high-complexity prompt to a low-tier model—results in a failed generation, which is heavily penalized. The thresholds are dynamically tuned until the validation FNR drops below 5%, ensuring the router defaults to a larger model when prediction uncertainty is high. The entire online routing decision executes in less than 5 ms and consumes under 0.05 J of electrical energy, representing less than 0.5% of the total energy of a standard 8B inference step[cite: 3].

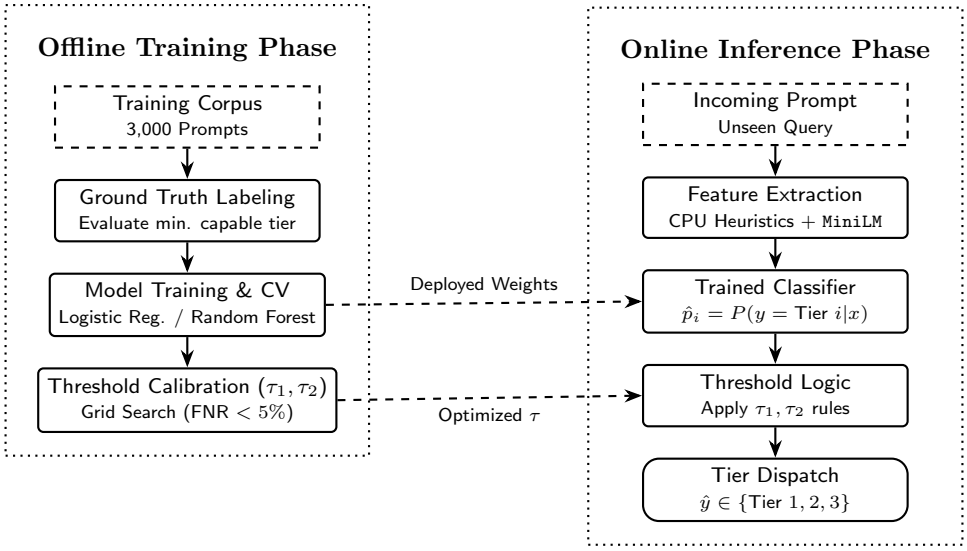

Figure 2: The two-phase pipeline of the prompt-complexity router. The offline phase establishes ground-truth labels and tunes the confidence thresholds (τ_1, τ_2) to minimize false negatives, dictating the logic for the sub-5 ms online inference phase.

3.3 Hardware Testbed, Energy Profiling, and Evaluation Protocol

To evaluate energy consumption across distinct silicon architectures and manufacturing process nodes under a uniform 8 GB VRAM budget, experiments are conducted across two dedicated consumer GPU platforms, as summarized in Table 2. Platform A is an NVIDIA GeForce RTX 4060, representing modern energy-efficient architecture built on the Ada Lovelace microarchitecture and fabricated on TSMC's 4N process node (115 W TDP, 3072 CUDA cores, and 8 GB GDDR6 VRAM). Platform B is an NVIDIA GeForce RTX 3060 Ti, representing previous-generation performance hardware built on the Ampere microarchitecture and fabricated on Samsung's 8nm process node (200 W TDP, 4864 CUDA cores, and 8 GB GDDR6 VRAM). Contrasting these two platforms enables a comparative analysis of how silicon manufacturing advancements, architectural tensor core scaling, and dynamic power gating interact with adaptive model routing under identical physical memory constraints.

Table 2: Hardware specifications and architectural characteristics of the evaluation GPU testbed.

Platform	GPU Model	Microarchitecture	CUDA Cores	VRAM	TDP
Platform A	NVIDIA GeForce RTX 4060	Ada Lovelace	3072	8 GB GDDR6	115 W
Platform B	NVIDIA GeForce RTX 3060 Ti	Ampere	4864	8 GB GDDR6	200 W

Dynamic hardware power draw $P(t)$ is sampled directly from onboard GPU sensors via the NVIDIA Management Library (`NVML`) using the `pynvml` Python bindings. A decoupled background thread queries instantaneous board power

at a sampling frequency of 50 ms (20 Hz). To prevent inference pipeline stalls, the profiler writes power samples to an in-memory ring buffer without issuing synchronous CUDA barrier calls. Total gross energy consumption E (in Joules) over the request execution window $[t_{\text{start}}, t_{\text{end}}]$ is calculated via numerical trapezoidal integration:

$$E = \sum_{k=1}^{N-1} \frac{P(t_k) + P(t_{k+1})}{2} \cdot (t_{k+1} - t_k) \tag{1}$$

To isolate the energy directly attributable to model execution from static peripheral power, the idle baseline power draw P_{idle} is profiled during steady-state quiescent intervals prior to each test batch and subtracted to yield the net dynamic inference energy:

$$E_{\text{net}} = E - \int_{t_{\text{start}}}^{t_{\text{end}}} P_{\text{idle}}\, dt \tag{2}$$

The end-to-end serving pipeline is evaluated against a curated benchmark workload comprising 600 distinct prompts partitioned equally across three operational complexity categories (200 prompts per tier). The low-complexity partition draws from the Stanford Sentiment Treebank (SST-2) for binary sentiment classification and SQuAD v2.0 for closed-domain factual retrieval, representing high-volume, low-cognitive-load queries. The medium-complexity partition draws from CNN/DailyMail for multi-sentence abstractive summarization and Multi-Genre Natural Language Inference (MNLI) for semantic entailment and deduction, requiring multi-sentence contextual comprehension. Finally, the high-complexity partition incorporates Grade School Math (GSM8K) for multi-step arithmetic reasoning and HumanEval for functional Python code synthesis, demanding rigorous symbolic deduction and algorithmic generation.

System performance is comprehensively evaluated across three primary dimensions: energy efficiency, output quality retention, and serving latency.

For energy efficiency, we assess both aggregate workload consumption and per-token efficiency. Total electrical energy consumed across the benchmark workload is measured as E_{total} (in kJ). To quantify fine-grained token-level efficiency independent of output length variation, we compute the normalized energy efficiency per generated output token:

$$\text{Joules/Token} = \frac{E}{N_{\text{tokens}}} \tag{3}$$

where E denotes the measured dynamic inference energy in Joules and N_{tokens} is the number of generated output tokens.

Furthermore, to quantify cross-generation hardware efficiency gains under adaptive routing, we compute the microarchitectural energy scaling ratio:

$$\eta = \frac{E_{\text{RTX 3060 Ti}}}{E_{\text{RTX 4060}}} \tag{4}$$

Output quality retention is assessed against ground-truth references using task-appropriate standard evaluation metrics: classification Accuracy and Exact Match (EM) for SST-2 and SQuAD, ROUGE-L F_1 score for CNN/DailyMail abstractive summarization, and Pass@1 execution accuracy for HumanEval code generation. To quantify how well the adaptive router preserves the performance of the largest foundation model, we define the Quality Preservation Ratio (QPR) as the percentage of task accuracy maintained relative to an unrouted, static Tier 3 baseline:

$$\text{QPR} = \frac{\text{Score}_{\text{Router}}}{\text{Score}_{\text{Tier 3 Static}}} \times 100\% \tag{5}$$

A QPR approaching 100% indicates that routing delivers significant energy savings with virtually no perceptual or functional penalty in generation fidelity.

Finally, the latency profile is evaluated by tracking Time-to-First-Token (TTFT), total End-to-End Latency (T_{e2e}), and isolated Router Overhead (Δt_{route}), ensuring that the energy-optimized routing mechanism does not compromise user-facing interactive responsiveness.

References

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy, 2019, pp. 3645–3650.

[2] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[3] S. Samsi, D. Zhao, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Bergeron, J. Kepner, D. Tiwari, and V. Gadepally, "From words to watts: Benchmarking the energy costs of large language model inference," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*, 2023, pp. 1–9.

[4] J. Fernandez, C. Na, V. Tiwari, Y. Bisk, S. Luccioni, and E. Strubell, "Energy considerations of large language model inference and efficiency optimizations," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria, 2025, pp. 32 556–32 569.

[5] G. Wilkins, S. Keshav, and R. Mortier, "Hybrid heterogeneous clusters can lower the energy consumption of LLM inference workloads," in *Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems (e-Energy '24)*, 2024, pp. 506–513.

[6] Z. Ji and M. Jiang, "A systematic review of electricity demand for large language models: Evaluations, challenges, and solutions," *Renewable and Sustainable Energy Reviews*, 2025.

[7] C. Niu, W. Zhang, J. Li, Y. Zhao, T. Wang, X. Wang, and Y. Chen, "TokenPowerBench: Benchmarking the power consumption of LLM inference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 38, 2026, pp. 32 582–32 590.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pre-trained transformers," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[9] P. Li, J. Yang, M. A. Islam, and S. Ren, "Making AI less "thirsty": Uncovering and addressing the secret water footprint of AI models," *Communications of the ACM*, vol. 68, no. 7, pp. 54–61, 2025.

[10] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*, 2023, pp. 38 087–38 099.

[11] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30 318–30 332.

[12] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *Transactions on Machine Learning Research*, 2024.

[13] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[14] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, "Hybrid LLM: Cost-efficient and quality-aware query routing," in *International Conference on Learning Representations (ICLR)*, 2024.

[15] T. Khan, S. Motie, S. A. Kocak, and S. Raza, "Optimizing large language models: Metrics, energy efficiency, and case study insights," in *2025 IEEE Conference on Artificial Intelligence (CAI)*, 2025, pp. 370–375.

[16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," in *Proceedings of Machine Learning and Systems*, vol. 6, 2024, pp. 87–100.

[17] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, "Estimating the carbon footprint of BLOOM, a 176b parameter language model," *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023.